

Introduction to Linked List Deletion

A linked list is a linear data structure where each element, called a node, contains a data value and a pointer to the next node in the sequence. Unlike arrays, linked list nodes are not stored in contiguous memory locations, which gives them flexibility in insertion and deletion operations.

Deletion in a Linked List

Deleting a node from a linked list requires careful pointer manipulation to maintain the integrity of the list. There are three main cases that must be handled when performing a deletion:

1. Deletion at the Head

When the node to be deleted is the first node (the head), the operation is straightforward. The head pointer is simply updated to point to the second node in the list. The original head node is then freed from memory. This is considered the simplest deletion case because no traversal is needed.

2. Deletion at the Tail

Deleting the last node requires traversing the entire list to reach the second-to-last node. Once found, its next pointer is set to null, effectively detaching the last node from the list. This case requires $O(n)$ time since the list must be traversed.

3. Deletion by Value or Index

The most general case involves finding and removing a node that is somewhere in the middle of the list. This requires two pointers: one that tracks the current node and one that tracks the previous node. When the target node is found, the previous node's next pointer is updated to skip over the target node and point directly to the node after it. This effectively removes the target from the chain without breaking the list structure.

Handling Edge Cases

Special attention must be paid to edge cases such as attempting to delete from an empty list, which should raise an error or return gracefully. Additionally, when deleting by value, if the value does not exist in the list, the list should remain unchanged.

Time Complexity

All three deletion operations run in $O(n)$ time in the worst case due to traversal, except for head deletion which runs in $O(1)$. Deletion does not require shifting elements as in arrays, which makes linked lists advantageous when frequent deletions are expected.